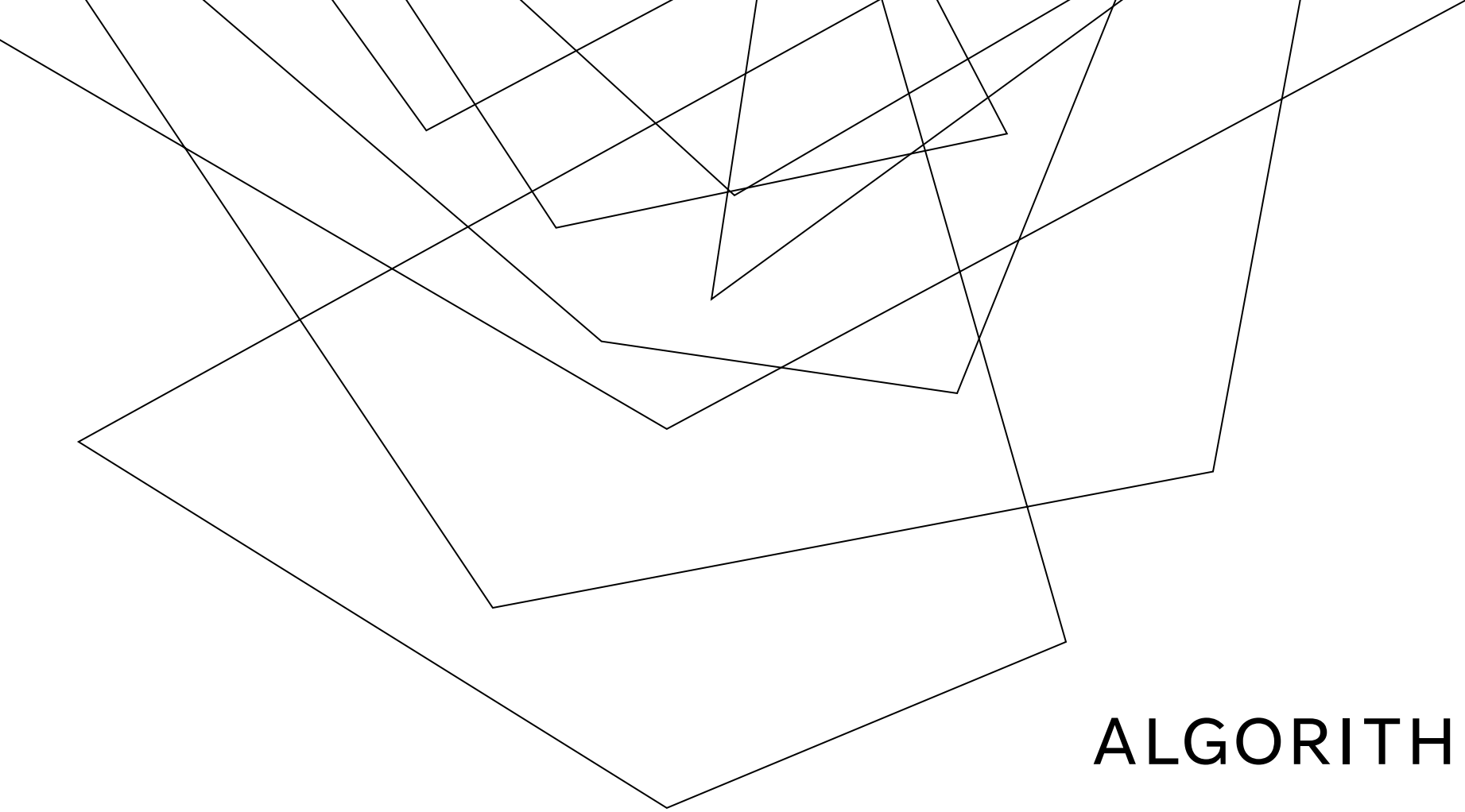


A series of thin, black, overlapping lines forming various geometric shapes like triangles and polygons, creating a complex, abstract pattern in the upper left portion of the slide.

ALGORITHMS

5. SORTING (2)





공지 사항

ASSIGNMENT #1 IS OUT

- Due : 9/17 (목) 23 : 59 (**TODAY**)
- 주의 : 제출 기한이 지난 후에는 추가 제출 (딜레이)은 받지 않음



PCCP 일정 변경

- 11/21 (토) → 11/14 (토) 14:00 – 16:00
- 슬랑제 일정으로 인하여 변경





지난 시간 복습

SORTING (1)

- 1. Sorting – overview**
- 2. Bubble sort**
- 3. Selection sort**
- 4. Insertion sort**





목차

목차

1. Merge sort

2. Quick sort



An abstract graphic design featuring two thin, dark gray lines that intersect on a light gray background. One line is oriented diagonally from the top-left towards the bottom-right, while the other is oriented from the top-right towards the bottom-left. The intersection point is located in the upper-left quadrant of the image. To the right of the intersection, the words "MERGE SORT" are written in a bold, black, sans-serif font, arranged in two lines: "MERGE" on the top line and "SORT" on the bottom line.

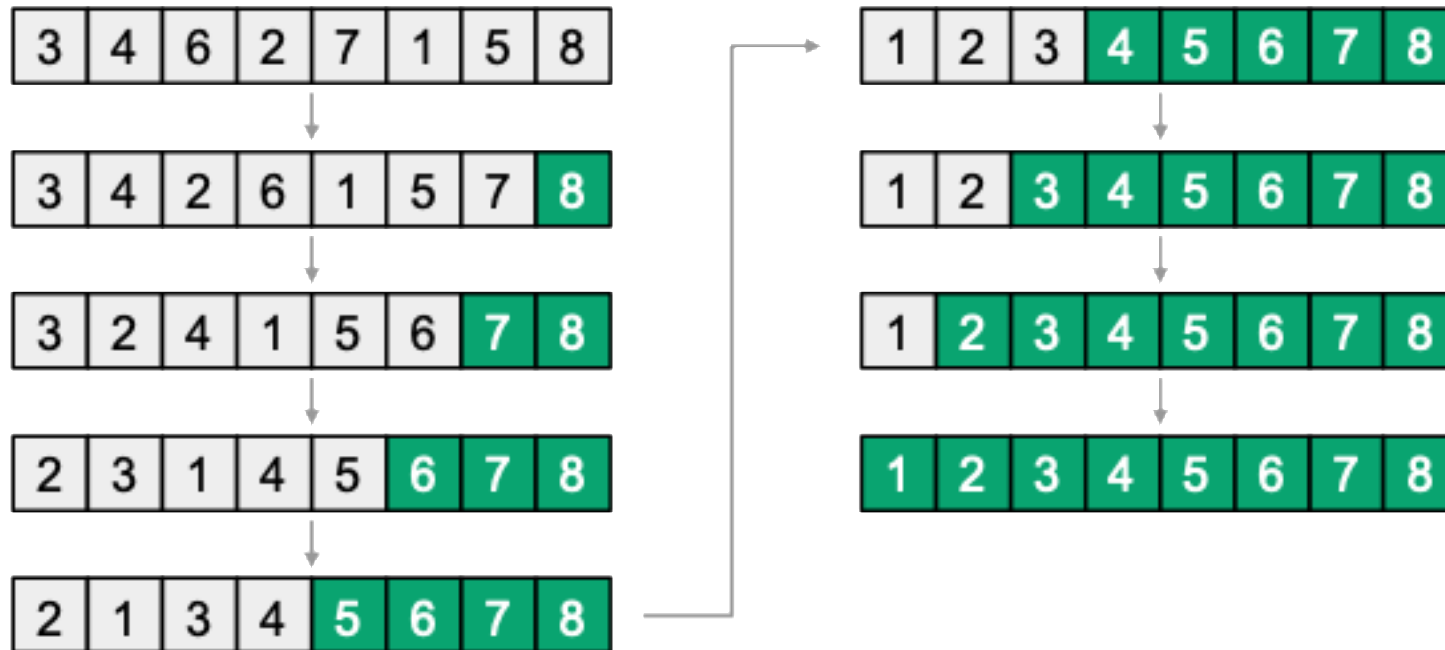
MERGE SORT

MERGE SORT

Cf) 지난 시간 Bubble sort, Selection sort, Insertion sort :

배열의 원소를 “한 개씩” 늘려가며 Divide and conquer

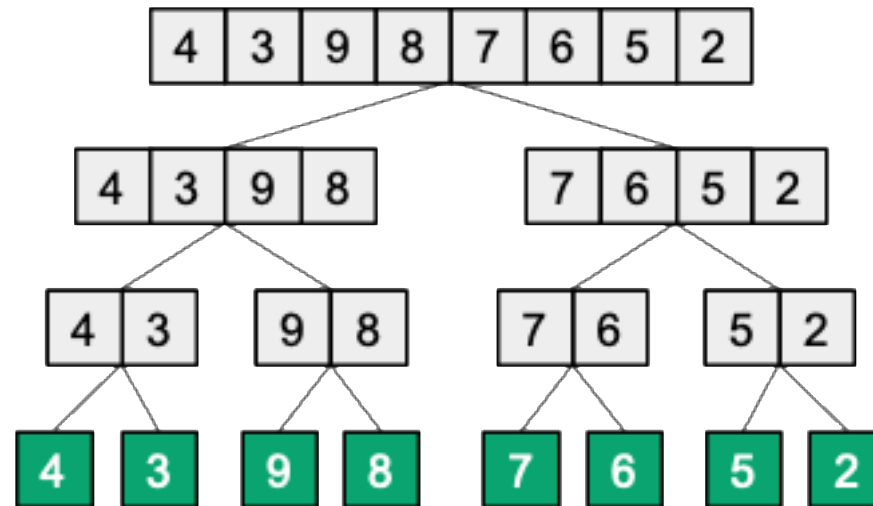
➔ 필요한 Divide의 회수가 $n \text{ step}$



MERGE SORT

Intuition) 원소 하나씩 대신, 배열을 “절반씩” 나눈 (Divide) 후 합치기 (Conquer)

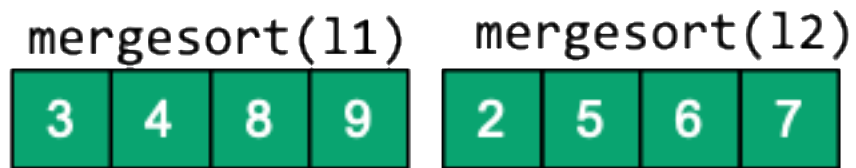
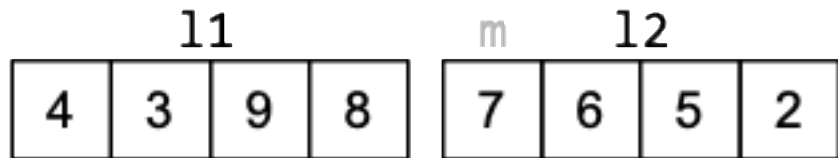
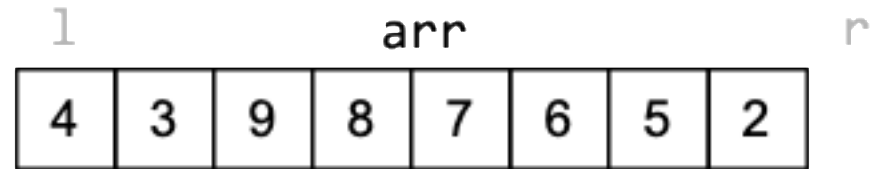
→ 필요한 Divide의 회수가 $\log n$ step으로 감소



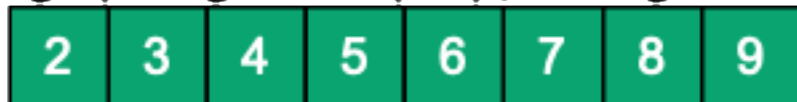
MERGE SORT

배열을 절반으로 나눠서 각각 정렬하고 합침 (merge)

mergesort - 재귀함수



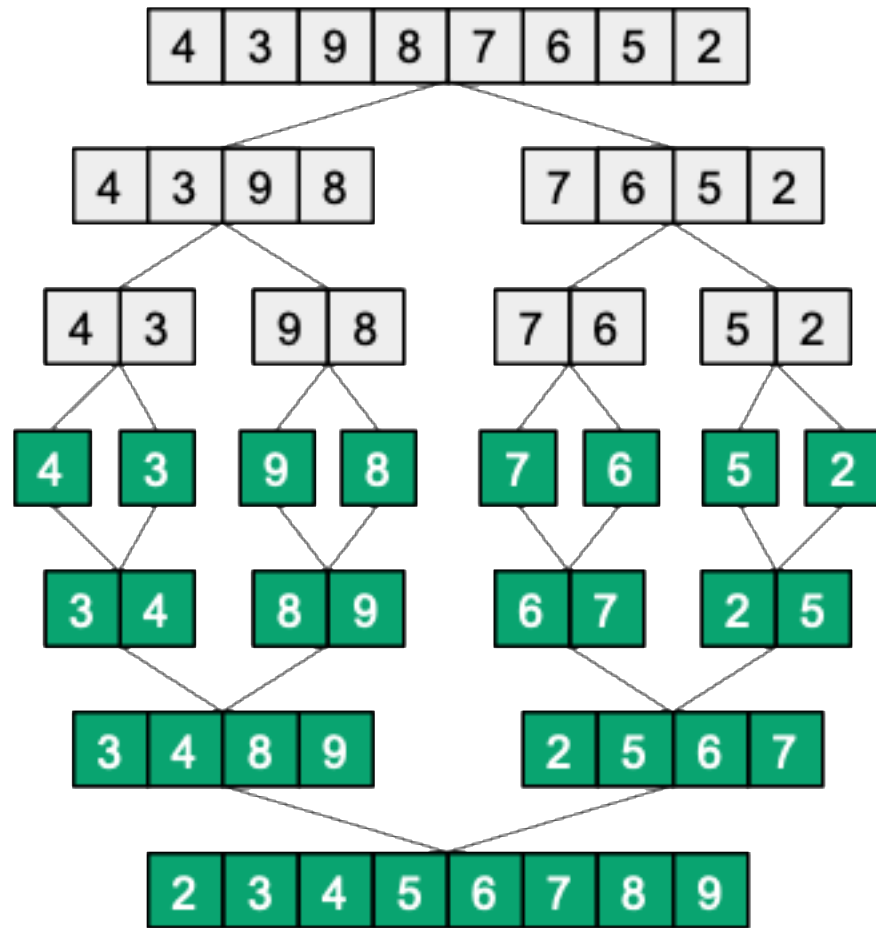
merge(mergesort(11), mergesort(12))



```
def mergesort(arr, l, r):  
    if l + 1 == r: return  
    m = (l + r) // 2  
    mergesort(arr, l, m)  
    mergesort(arr, m, r)  
    merge(arr, l, m, r)
```



MERGE SORT



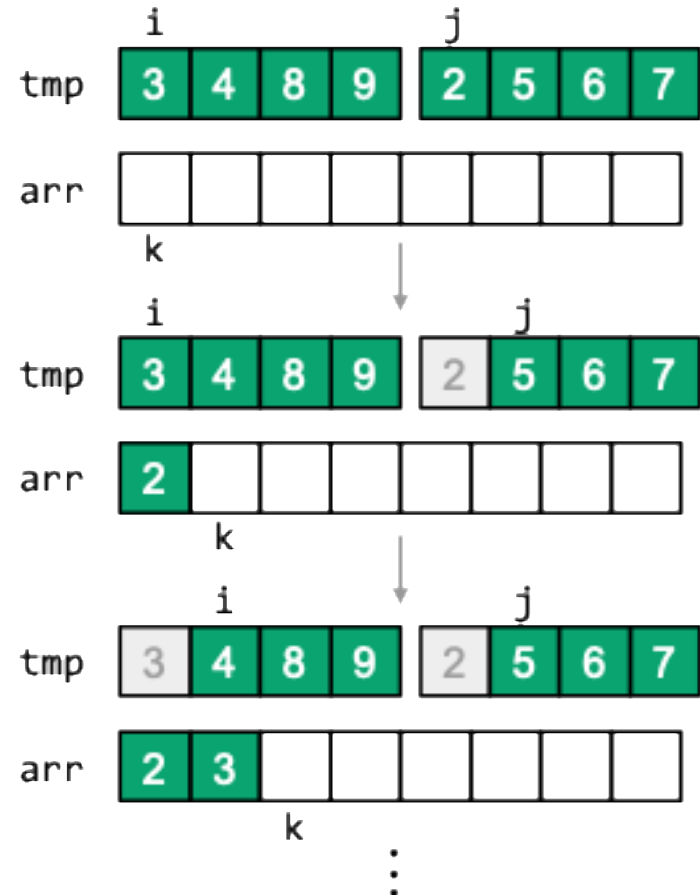
```
def mergesort(arr, l, r):  
    if l + 1 == r: return  
    m = (l + r) // 2  
    mergesort(arr, l, m)  
    mergesort(arr, m, r)  
    merge(arr, l, m, r)
```



MERGE SORT

merge : 두개의 작은 배열을 하나의 큰 배열로 합치는 함수

두 작은 배열 각각의 최솟값을 비교, 더 작은 값을 복사해옴



```
def merge(arr, l, m, r):  
    tmp = arr[l:r]  
    i, j, k = 0, m-1, l  
    while i < (m-1) and j < (r-1):  
        if tmp[i] < tmp[j]:  
            arr[k] = tmp[i]  
            i += 1; k += 1  
        else:  
            arr[k] = tmp[j]  
            j += 1; k += 1  
    while i < m-1:  
        arr[k] = tmp[i]  
        i += 1; k += 1  
    while j < r-1:  
        arr[k] = tmp[j]  
        j += 1; k += 1
```



MERGE SORT

Animation

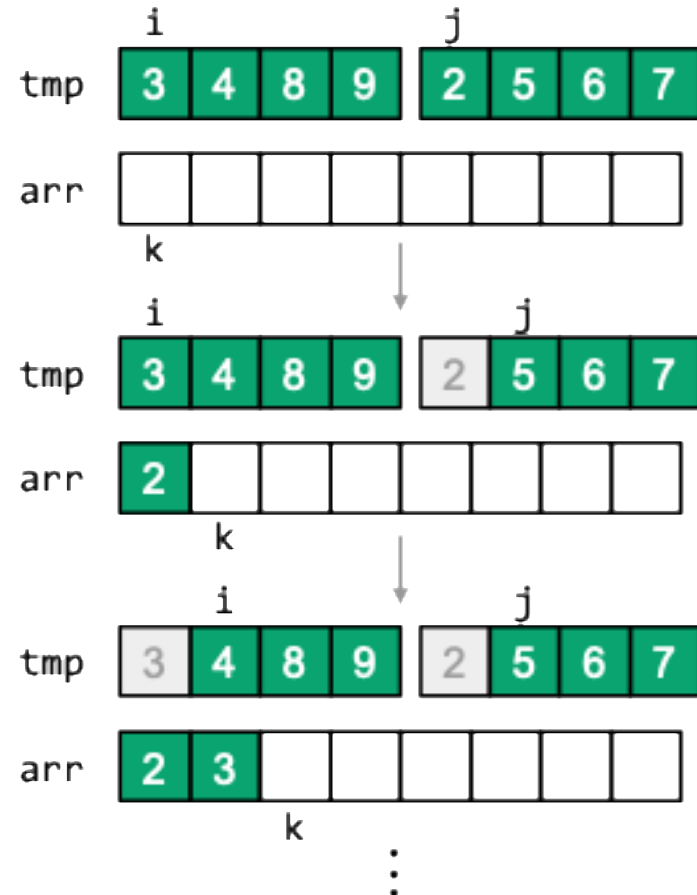
6 5 3 1 8 7 2 4



MERGE SORT

Complexity analysis

1) merge 함수



```
def merge(arr, l, m, r):  
    tmp = arr[l:r]  
    i, j, k = 0, m-1, l  
    while i < (m-1) and j < (r-1):  
        if tmp[i] < tmp[j]:  
            arr[k] = tmp[i]  
            i += 1; k += 1  
        else:  
            arr[k] = tmp[j]  
            j += 1; k += 1  
    while i < m-1:  
        arr[k] = tmp[i]  
        i += 1; k += 1  
    while j < r-1:  
        arr[k] = tmp[j]  
        j += 1; k += 1
```

MERGE SORT

Complexity analysis

2) mergesort 함수

```
def mergesort(arr, l, r):  
    if l + 1 == r: return  
    m = (l + r) // 2  
    mergesort(arr, l, m)  
    mergesort(arr, m, r)  
    merge(arr, l, m, r)
```

➔ Asymptotic analysis ?



The image features a minimalist design on a light gray background. Two thin, dark gray lines intersect: one line slopes downward from the top-left towards the bottom-right, and the other slopes downward from the top-right towards the bottom-left. To the right of this intersection, the words "QUICK SORT" are written in a bold, black, sans-serif font.

QUICK SORT

QUICK SORT

Intuition) 정렬이 완료된 시점을 기준으로

“특정 원소 시점에서” 왼쪽 원소들은 모두 자신보다 값이 작고
오른쪽 원소들은 모두 자신보다 값이 큼

특정 원소 기준으로 배열을 좌우로 둘로 나눠 생각할 수 있음



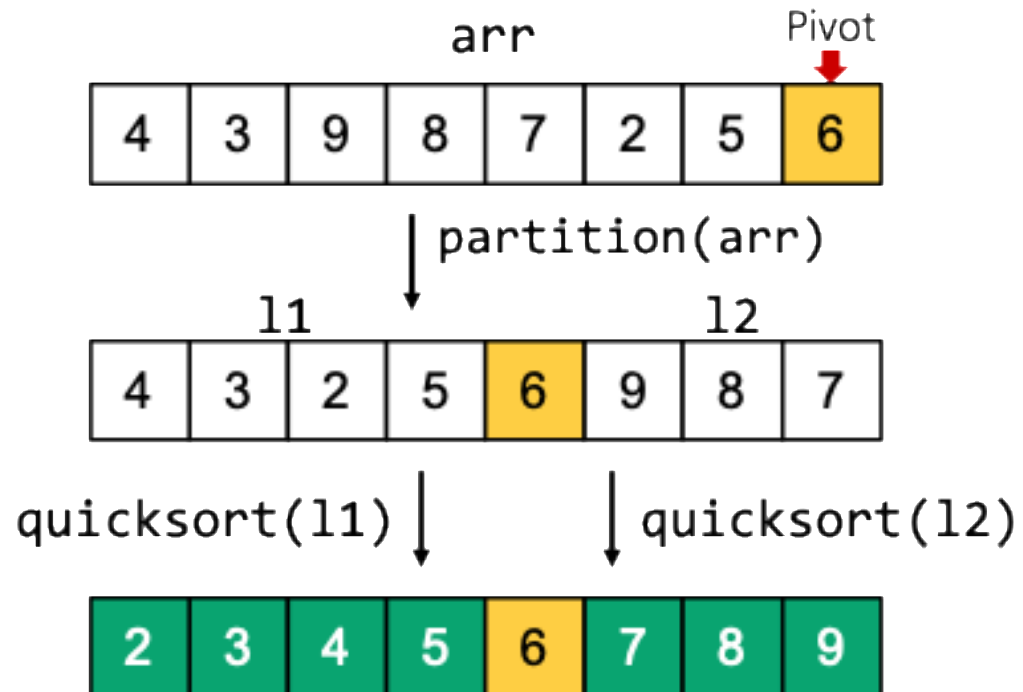
QUICK SORT

배열의 맨 오른쪽 원소를 Pivot으로 설정

Pivot 원소를 올바른 위치로 옮기고,

Pivot을 기준으로 배열을 둘로 나눠서 각각 정렬

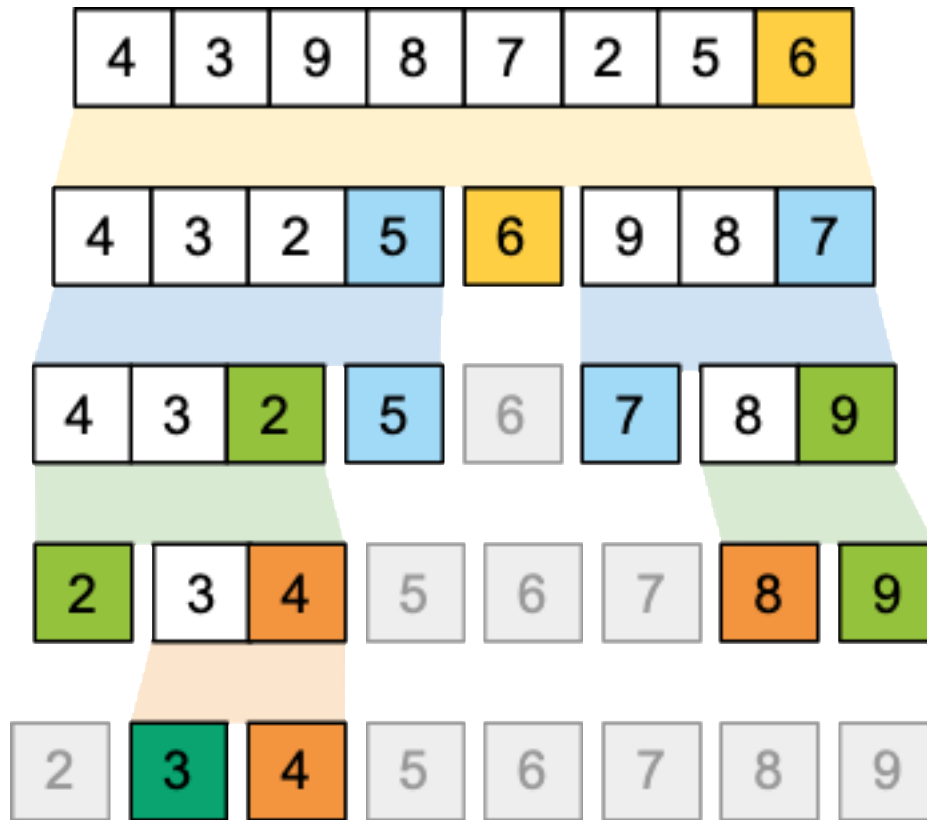
quicksort - 재귀함수



```
def quicksort(arr, l, r):  
    if l + 1 >= r: return  
    p = partition(arr, l, r)  
    quicksort(arr, l, p)  
    quicksort(arr, p+1, r)
```



QUICK SORT



```
def quicksort(arr, l, r):  
    if l + 1 >= r: return  
    p = partition(arr, l, r)  
    quicksort(arr, l, p)  
    quicksort(arr, p+1, r)
```

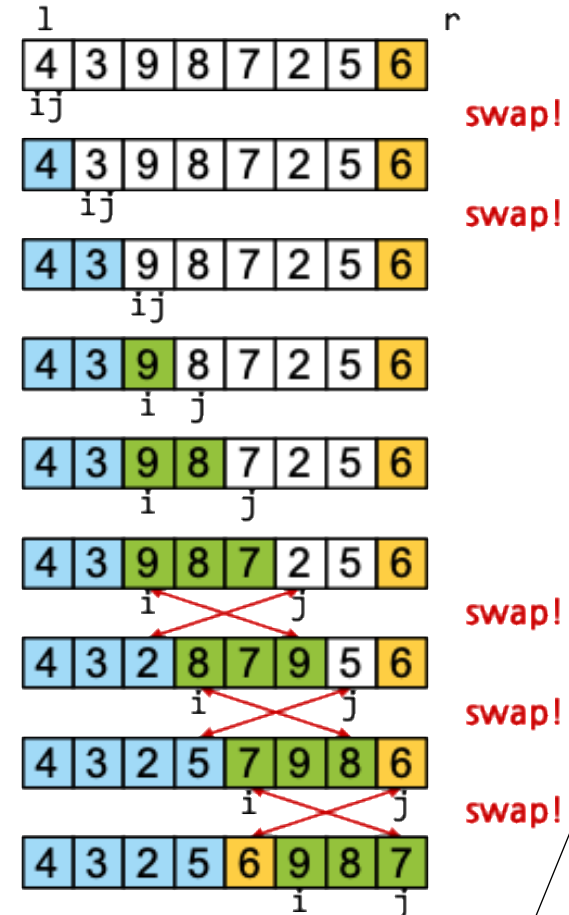
QUICK SORT

partition: Lomuto가 제안한 분할 알고리즘

pivot 값을 기준으로 왼쪽에는 더 작은 원소들이,
오른쪽에는 더 큰 원소들이 오도록 정렬

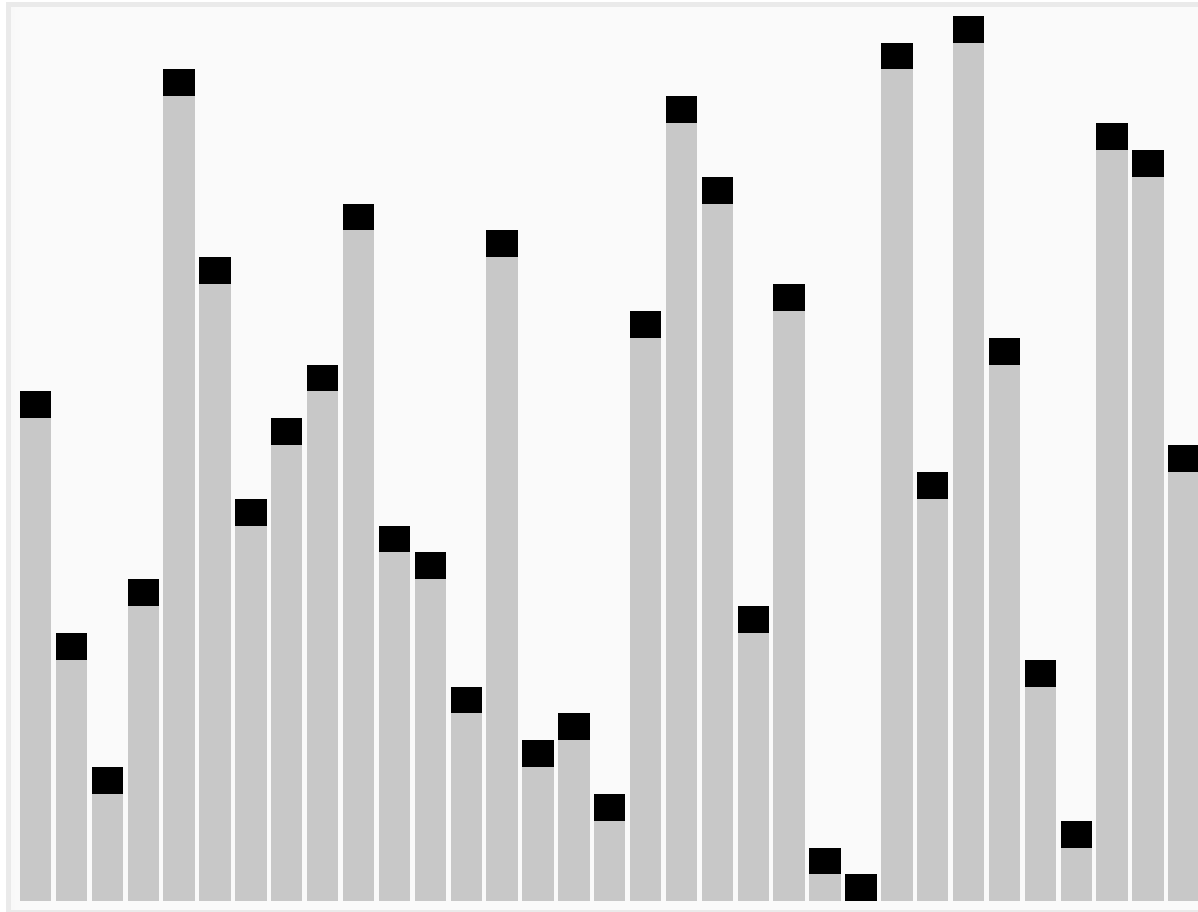
```
def partition(arr, l, r):  
    p = arr[r-1] # pivot  
    i = l  
    for j in range(l, r):  
        if arr[j] <= p:  
            arr[i], arr[j] = arr[j], arr[i]  
            i += 1  
    return i-1
```

구현 방법은 여러가지가 존재



QUICK SORT

Animation

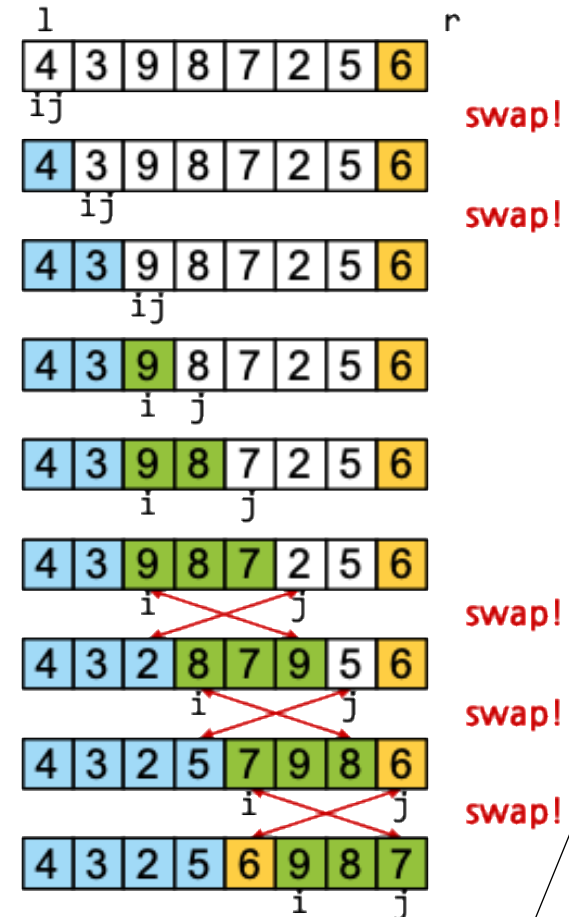


QUICK SORT

Complexity analysis

1) partition 함수

```
def partition(arr, l, r):  
    p = arr[r-1] # pivot  
    i = l  
    for j in range(l, r):  
        if arr[j] <= p:  
            arr[i], arr[j] = arr[j], arr[i]  
            i += 1  
    return i-1
```



QUICK SORT

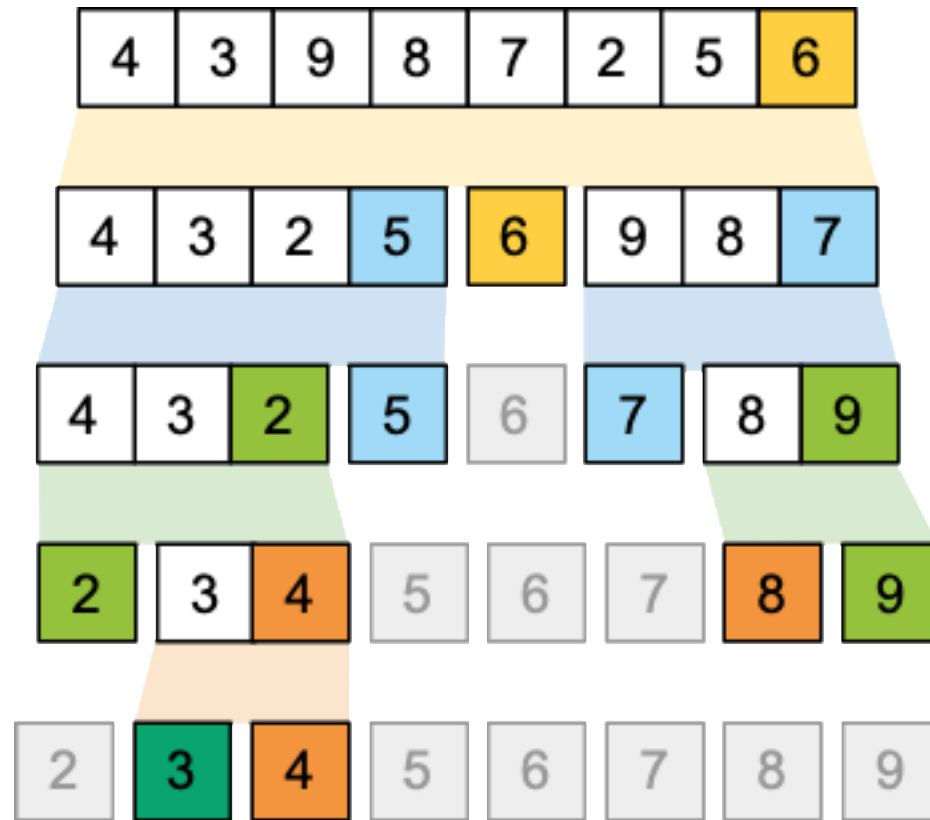
Complexity analysis

2) quicksort 함수

재귀호출 양상의 차이

- Pivot 값이 최댓값 혹은 최솟값 :
- 그렇지 않은 경우 :

```
def quicksort(arr, l, r):  
    if l + 1 >= r: return  
    p = partition(arr, l, r)  
    quicksort(arr, l, p)  
    quicksort(arr, p+1, r)
```



QUICK SORT

Complexity analysis

2) quicksort 함수

- Best case : partition 함수가 항상 배열을 절반으로 나누는 경우
- Worst case : Pivot 값이 배열에서 항상 가장 크거나 가장 작은 경우



QUICK SORT

Complexity analysis

2) quicksort 함수

- Average case : 모든 경우의 평균
pivot의 위치가 균등함을 가정



An abstract graphic on a light gray background. Two thin, dark gray lines intersect. One line is nearly vertical, starting from the top center and extending towards the bottom right. The other line is nearly horizontal, starting from the top left and extending towards the middle right. The intersection point is located in the upper-left quadrant of the image. To the right of the intersection, the word "SUMMARY" is written in a bold, black, sans-serif font.

SUMMARY

SUMMARY

1. Merge sort

2. Quick sort



NEXT LECTURE

Sorting (3) - Advanced sorting algorithm – Heap sort



Two thin, dark gray lines intersect diagonally on a light gray background. One line runs from the top-left towards the bottom-right, and the other runs from the top-right towards the bottom-left. They cross each other in the upper-left quadrant of the image.

QUESTIONS?